Applied Statistics
Graded Assignment 2

Name: Faran Taimoor Butt

Email: faranbutt@phystech.edu

1 You can access the Colab notebook for codes.

1.1 Problem 1

1. (2 points) Suppose you have the following sample:

Interval (x)	0	1	2	3	4	5	6	7	8	9
Count (O_i)	74	92	83	79	80	73	77	75	76	91

Use the Pearson χ^2 test to test the hypothesis at $\alpha = 0.05$ level:

$$H_0 : X \sim \text{Uniform}[0, 9] \quad \text{vs.} \quad H_1 : X \not\sim \text{Uniform}[0, 9]$$

Solution:

Using Explicit Formulas:

```
[ ] import numpy as np
    import scipy.stats as sts

[ ] X = np.array([74,92,83,79,80,73,77,75,76,91
    ])
    n = X.sum()
    r = len(X)

[ ] alpha = 0.05
    p = 1/10

[ ] chisq = sts.chi2(r-1)
    c = chisq.isf(alpha)
    c

    np.float64(16.91897760462045)

[ ] T = np.sum((X - n*p)**2 / (n*p))

    p_value = chisq.sf(T)
    print(f"P_Value = {p_value}")

    P_Value = 0.8232783432788753

[ ] if p_value < alpha:
      print("Rejected H_0: The sample dosent belong to Uniform distribtuion")
    else:
      print("Failed to Reject H_0: The sample may be from Uniform distribution")

    Failed to Reject H_0: The sample may be from Uniform distribution
```

Figure 1:

Using Library:

```
_,p_value = sts.chisquare(X)
print(f"P Value = {p_value}")
if p_value < alpha:
   print("Rejected H_0: The sample dosent belong to Uniform distribtuion")
else:
  print("Failed to Reject H_0: The sample may be from Uniform distribution")
```

```
P Value = 0.8232783432788753
Failed to Reject H_0: The sample may be from Uniform distribution
```

Figure 2:

2. (1 point) **1.2 Problem 2**

Suppose you have the following two independent samples, $X \sim N(\mu_x, \sigma^2)$ and $Y \sim N(\mu_y, \sigma^2)$ (see Table 1). Use Student's two-sample t-test to test the following hypothesis at $\alpha = 0.05$ level. Assume equal variances.

$$H_0 : \mu_x = \mu_y \quad \text{vs.} \quad H_1 : \mu_x \neq \mu_y$$

Data:

Index	X	Y
1	-1.75	-0.29
2	-0.33	0.09
3	-1.26	1.70
4	0.32	-1.09
5	1.53	-0.44
6	0.35	-0.29
7	-0.96	0.25
8	-0.06	-0.54
9	0.42	-1.38
10	-1.08	0.32

Solution: We will use the formula with equal variance for X and Y to solve manually

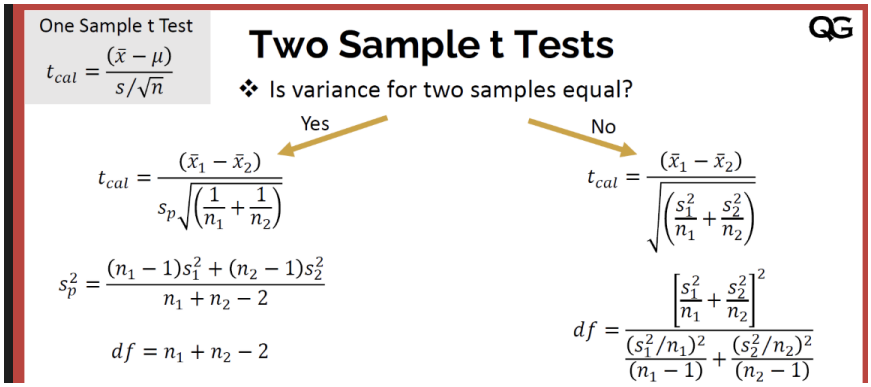


Figure 3:

Manually using code:

```
[ ] from scipy.stats import ttest_ind,t

[ ] X = np.array([-1.75, -0.33, -1.26, 0.32, 1.53, 0.35, -0.96, -0.06, 0.42, -1.08])
    Y = np.array([-0.29, 0.09, 1.70, -1.09, -0.44, -0.29, 0.25, -0.54, -1.38, 0.32])

[ ] n = len(X)
    m = len(Y)
    X_hat = X.mean()
    Y_hat = Y.mean()

[ ] X_var = X.var(ddof=1)
    Y_var = Y.var(ddof=1)

[ ] #pool vairance
    s_p = np.sqrt(((n-1) * X_var**2 + (m-1) * Y_var**2) / (n + m - 2))
    s_p

    np.float64(0.8607995585274839)

    T_stat = (X_hat - Y_hat) / (s_p * np.sqrt(1/n + 1/m))
    T_stat

    np.float64(-0.29873135373391996)

[ ] df = n + m - 2
    p_value = 2 * t.sf(abs(T_stat),df) # two tailed thats why multiply by 2
    p_value

    np.float64(0.7685663931031849)

[ ] print('Rejected Null Hypothesis u_x != u_y' if p_value < alpha else "Not Rejected Null Hyphothesis u_x = u_y" )

    Not Rejected Null Hyphothesis u_x = u_y
```

Figure 4:

Verifying using the Library:

```
[ ] X = np.array([-1.75, -0.33, -1.26, 0.32, 1.53, 0.35, -0.96, -0.06, 0.42, -1.08])
    Y = np.array([-0.29, 0.09, 1.70, -1.09, -0.44, -0.29, 0.25, -0.54, -1.38, 0.32])
    alpha = 0.05

    t_stat,p_value = ttest_ind(X,Y,equal_var=True)
    print(f"T-statistic: {t_stat:.4f}")
    print(f"P-value: {p_value:.4f}")
    print('Rejected Null Hypothesis u_x != u_y' if p_value < alpha else "Not Rejected Null Hyphothesis u_x = u_y" )

    T-statistic: -0.2787
    P-value: 0.7837
    Not Rejected Null Hyphothesis u_x = u_y
```

Figure 5:

3. (4 points) **1.3 Problem 3**

Let $X_1, \ldots, X_n \sim N(\theta, 1)$. Consider the test:

$$H_0 : \theta = \theta_0 = 0 \quad \text{vs.} \quad H_1 : \theta = \theta_1 = 1$$

Let the rejection region be defined as:

$$R = \{X : T(X) > c\}, \quad \text{where} \quad T(X) = \frac{1}{n}\sum_{i=1}^{n} X_i = \bar{X}$$

4. (2 points) Find c so that the test has size α

Solution:

$$P_{\theta=0}(\bar{X} > c) = \alpha$$

$$P\left(Z > \frac{c-0}{1/\sqrt{n}}\right) = \alpha \quad \Rightarrow \quad \frac{c\sqrt{n}}{1} = z_{1-\alpha}$$

$$c = \frac{z_{1-\alpha}}{\sqrt{n}}$$

5. (2 points) Find the test power β

Solution:

$$\beta = P_{\theta=1}(\bar{X} > c)$$

$$\beta = P\left(Z > \frac{c-1}{1/\sqrt{n}}\right) = P\left(Z > \sqrt{n}(c-1)\right)$$

$$c = \frac{z_{1-\alpha}}{\sqrt{n}} \quad \Rightarrow \quad \sqrt{n}(c-1) = z_{1-\alpha} - \sqrt{n}$$

$$\beta = P\left(Z > z_{1-\alpha} - \sqrt{n}\right) = 1 - \Phi(z_{1-\alpha} - \sqrt{n})$$

$$\beta = 1 - \Phi(z_{1-\alpha} - \sqrt{n})$$

6. (4 points) **1.4 Problem 4**

Consider the following situation. You have an anti-theft alarm a installed in your apartment. It is a good alarm, and it is triggered when a thief t breaks into your apartment. However it may also be triggered if an earthquake e happens. Finally, earthquakes are sometimes announced on the radio r.

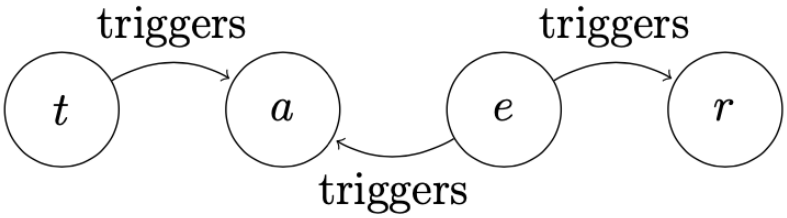


Figure 6:

We may write the following probabilistic model:

$$p(t, e, a, r) = p(a|t, e)p(r|e)p(t)p(e)$$

Define the following probabilities:

$p(a = 1\|t, e)$	$t = 0$	$t = 1$
$e = 0$	0	1
$e = 1$	0.1	1

	$e = 0$	$e = 1$
$p(r = 1\|e)$	0	0.5

And also define the probabilities $p(t = 1) = 2 \cdot 10^{-4}$ and $p(e = 1) = 10^{-2}$. Now compute the following:

- (2 points) Probability that there is a thief in your apartment if there is an alarm: $p(t = 1|a = 1)$

Solution:
As

$$p(t = 1) = 2 \times 10^{-4}$$
$$p(e = 1) = 10^{-2}$$

$$p(t = 1|a = 1) = \frac{p(t = 1, a = 1)}{p(a = 1)}$$
$$= \frac{\sum p(a = 1 \mid t = 1, e)p(e)p(t = 1)}{\sum p(a = 1 \mid t = 1, e)p(t)p(e)}$$

Solving top part

$$\sum p(a = 1 \mid t = 1, e)p(e)p(t = 1)$$

$$e = 0$$
$$p(a = 1 \mid t = 1, e = 0) = 1$$
$$p(e = 0) = 0.99$$

$$1 \times 0.99 \times 2 \times 10^{-4} = 1.98 \times 10^{-4}$$

$$e = 1$$
$$p(a = 1 \mid t = 1, e = 1) = 1$$
$$p(e = 1) = 10^{-2}$$

$$1 \times 0.01 \times 2 \times 10^{-4} = 2 \times 10^{-6}$$

total is

$$1.98 \times 10^{-4} + 2 \times 10^{-6} = 2 \times 10^{-4}$$

Bottom part:

$$\sum p(a = 1 \mid t, e)p(t)p(e)$$

lets take
$t = 1, e = 0$:

$$1 \times 2 \times 10^{-4} \times 0.99 = 1.98 \times 10^{-4}$$

$t = 1, e = 1$

$$1 \times 2 \times 10^{-4} \times 0.01 = 2.0 \times 10^{-6}$$

$t = 0, e = 0$

$$0 \times 0.9998 \times 0.99 = 0$$

$t = 0, e = 1$

$$0.1 \times 0.9998 \times 0.01 = 9.998 \times 10^{-4}$$

so

$$1.98 \times 10^{-4} + 2 \times 10^{-6} + 0 + 9.998 \times 10^{-4} = 1.17 \times 10^{-3}$$

$$\begin{aligned} p(t = 1 \mid a = 1) &= \frac{p(t = 1 \cap a = 1)}{p(a = 1)} \\ &= \frac{2.00 \times 10^{-4}}{1.17 \times 10^{-3}} \\ &\approx 0.169 \end{aligned}$$

- (2 points) Probability that there is a thief in your apartment if there is an alarm and you hear an announcement about an earthquake on the radio: $p(t = 1|a = 1, r = 1)$

Solution:

$$p(t = 1a = 1, r = 1) = ?$$

$$p(t=1 \mid a=1, r=1) = \frac{p(t=1, a=1, r=1)}{p(a=1, r=1)}$$
$$= \frac{\sum_e p(a=1 \mid t=1, e)p(r=1 \mid e)p(t=1)p(e)}{\sum_{t,e} p(a=1 \mid t, e)p(r=1 \mid e)p(t)p(e)}$$

Top of dividend:
For e = 0:

$$p(r=1|e=0) = 0$$

e = 1:

$$1 \times 0.5 \times 0.01 \times 2 \times 10^{-4} = 1 \times 10^{-6}$$

Bottom of dividend: For t = 1,e=1

$$1 \times 0.5 \times 0.01 \times 2 \times 10^{-4} = 1 \times 10^{-6}$$

t=0,e=1

$$0.1 \times 0.5 \times 0.01 \times 0.9998 = 4.999 \times 10^{-5}$$

e=0:

$$p(r=1 \mid e=0) = 0$$

total =

$$1 \times 10^{-6} + 4.999 \times 10^{-5} = 5.099 \times 10^{-5}$$

$$p(t=1 \mid a=1, r=1) = \frac{1 \times 10^{-6}}{5.099 \times 10^{-5}} = 0.019$$

7. (6 points) **Problem 5**

Computer Experiment Use the following code to load the data and get acquainted with it:

```
from sklearn.datasets import load_wine
data = load_wine(as_frame=True)
df = data["frame"]
target = data["target"]
```

This dataset includes ten features (age, sex, body mass index, average blood pressure, and six blood serum measurements) obtained from $n = 442$ diabetes patients. The target is a quantitative measure of disease progression one year after baseline.

1.

Compute Pearson correlations between different features and the target. Pick one feature that has the highest absolute (positive or negative) correlation. (For inspiration, refer to the Webinar 8 notebook.)

Solution:

```
[ ] from sklearn.datasets import load_diabetes
    from sklearn.metrics import median_absolute_error,mean_squared_error

[ ] data = load_diabetes(as_frame=True)
    df = data["frame"]
    target = data["target"]

[ ] df
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0
...	...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207	178.0
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485	104.0
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491	132.0
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930	220.0
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064	57.0

442 rows × 11 columns

Figure 7:

```
[ ] corrs = df.corr(numeric_only=True)["target"].drop('target')
    corrs
```

	target
age	0.187889
sex	0.043062
bmi	0.586450
bp	0.441482
s1	0.212022
s2	0.174054
s3	-0.394789
s4	0.430453
s5	0.565883
s6	0.382483

dtype: float64

```
[ ] corrs = corrs.sort_values(key=abs,ascending=False)
    corrs
```

	target
bmi	0.586450
s5	0.565883
bp	0.441482
s4	0.430453
s3	-0.394789
s6	0.382483
s1	0.212022
age	0.187889
s2	0.174054
sex	0.043062

dtype: float64

```
[ ] top_feature = corrs.index[0]
    top_corr = corrs.iloc[0]
    top_feature, top_corr
('bmi', np.float64(0.5864501344746885))
```


Figure 8:

2.

Define:

X1 = df[<YOUR SELECTED FEATURE>]
Y = target

- Build a linear regression model to predict Y from X_1. Summarize your analysis, including a plot of the data with the fitted line.
- Repeat your analysis with:

  X2 = np.log(df[<YOUR SELECTED FEATURE>])

- Compare the results using Mean Squared Error (MSE) and Mean Absolute Error (MAE).

Solution:

```
[ ] from sklearn.linear_model import LinearRegression
    from sklearn.metrics import mean_squared_error,mean_absolute_error
    from sklearn.model_selection import train_test_split

[ ] X1 = df[[top_feature]]
    y = target

[ ] X1_train,X1_test,y_train,y_test = train_test_split(X1,y,test_size=0.2,random_state=42)

[ ] model = LinearRegression()
    model.fit(X1_train,y_train)
    y_pred = model.predict(X1_test)

[ ] plt.scatter(X1_test, y_test, alpha=0.5, label="Data")
    plt.plot(X1_test, y_pred, color="red", label="Fitted line")
    plt.xlabel("BMI")
    plt.ylabel("Disease Progression")
    plt.title("Linear Regression: Target vs BMI")
    plt.legend()
    plt.show()
```

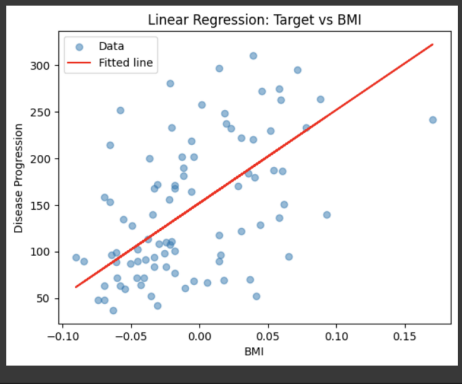
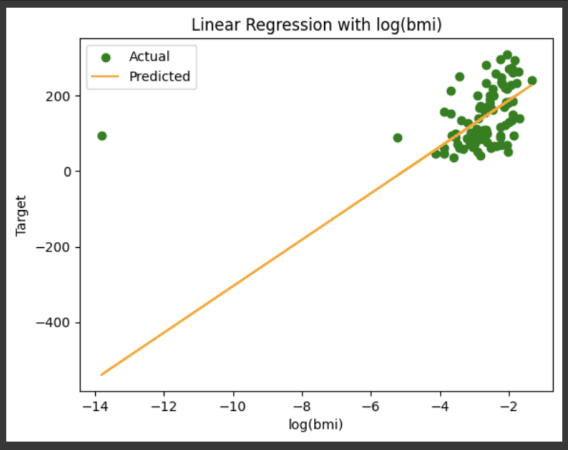


Figure 9:

```
[ ] mse1 = mean_squared_error(y_test, y_pred)
    mae1 = mean_absolute_error(y_test, y_pred)
    print(f"MSE = {mse1}")
    print(f"MAE = {mae1}")

    MSE = 4061.8259284949268
    MAE = 52.25997645345536

[ ] X2 = np.log(df[[top_feature]] + 1e-6 - df[top_feature].min())
    X2_train, X2_test,y_train,y_test = train_test_split(X2,y,test_size=0.2,random_state=42)

[ ]
    model2 = LinearRegression()
    model2.fit(X2_train,y_train)
    y_pred2 = model2.predict(X2_test)

[ ] mse2 = mean_squared_error(y_test,y_pred2)
    mae2 = mean_absolute_error(y_test,y_pred2)
    print(f"MSE with log = {mse2}")
    print(f"MAE with log= {mae2}")

    MSE with log = 8619.94406966433
    MAE with log= 60.009611283634655
```

Figure 10:

```
[ ] plt.scatter(X2_test, y_test, color='green', label='Actual')
    plt.plot(X2_test, y_pred2, color='orange', label='Predicted')
    plt.title(f'Linear Regression with log({top_feature})')
    plt.xlabel(f'log({top_feature})')
    plt.ylabel('Target')
    plt.legend()
    plt.show()
```




```
[ ] {
        "Top Feature": top_feature,
        "Correlation with Target": top_corr,
        "Original Feature MSE": mse1,
        "Original Feature MAE": mae1,
        "Log Feature MSE": mse2,
        "Log Feature MAE": mae2
    }

    {'Top Feature': 'bmi',
     'Correlation with Target': np.float64(0.5864501344746885),
     'Original Feature MSE': 4061.8259284949268,
     'Original Feature MAE': 52.259976445345536,
     'Log Feature MSE': 8619.94406966433,
     'Log Feature MAE': 60.009611283634655}
```

Figure 11:

3

Take all possible subsets of features using the following code:

```
from itertools import combinations, chain

def get_all_subsets(array):
    all_subsets = []
    for r in range(1, len(array) + 1):
        all_subsets.append(combinations(array, r))
    return list(chain(*all_subsets))
```

This should generate 1023 feature sets for 10 features.

- For each feature set, fit a linear regression model to predict Y.
- Compute and save the Akaike Information Criterion (AIC) of each model.
- Identify the best model in terms of AIC and report its MSE and MAE.

Solution:

Problem 5 - 3:

```
[ ] from itertools import combinations, chain
    import statsmodels.api as sm
```

```
[ ] def get_all_subsets(array):
        all_subsets = []
        for r in range(1, len(array) + 1):
            all_subsets.extend(combinations(array, r))
        return all_subsets
```

```
[ ] X_train,X_test,y_train,y_test = train_test_split(df.drop(columns = ['target']),y,test_size=0.2, random_state=42)
```

```
[ ] X1_train = X_train[[top_feature]]
    X1_test  = X_test[[top_feature]]
```

```
[ ] lr1 = LinearRegression()
    lr1.fit(X1_train, y_train)
    y1_pred = lr1.predict(X1_test)
```

```
[ ] plt.figure(figsize=(6,4))
    plt.scatter(X1_test, y_test, alpha=0.6, label="Actual")
    plt.plot(X1_test, y1_pred, color="red", label="Fitted line")
    plt.xlabel(top_feature)
    plt.ylabel("Disease Progression")
    plt.title(f"Linear Regression on {top_feature}")
    plt.legend()
    plt.show()
```

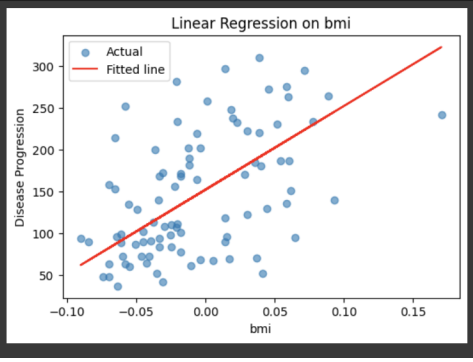


Figure 12:

```
[ ] mse1 = mean_squared_error(y_test, y1_pred)
    mae1 = mean_absolute_error(y_test, y1_pred)

    print(f"Original {top_feature} → MSE: {mse1:.2f}, MAE: {mae1:.2f}")

→ Original bmi → MSE: 4061.83, MAE: 52.26

[ ] min_val = df[top_feature].min()
    shift   = 1e-6 - min_val
    X2 = np.log(df[[top_feature]] + shift)

[ ] X2_train = X2.loc[X1_train.index]
    X2_test  = X2.loc[X1_test.index]

[ ] lr2 = LinearRegression()
    lr2.fit(X2_train, y_train)
    y2_pred = lr2.predict(X2_test)

[ ] plt.figure(figsize=(6,4))
    plt.scatter(X2_test, y_test, alpha=0.6, label="Actual")
    plt.plot(X2_test, y2_pred, color="orange", label="Fitted line")
    plt.xlabel(f"log({top_feature})")
    plt.ylabel("Disease Progression")
    plt.title(f"Linear Regression on log({top_feature})")
    plt.legend()
    plt.show()
```

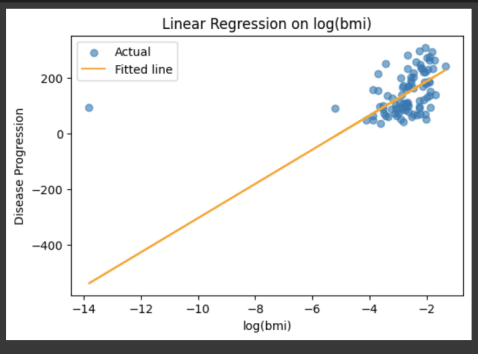


Figure 13: Enter Caption

```
[ ] mse2 = mean_squared_error(y_test, y2_pred)
    mae2 = mean_absolute_error(y_test, y2_pred)

    print(f"log({top_feature}) → MSE: {mse2:.2f}, MAE: {mae2:.2f}")

→ log(bmi) → MSE: 8619.94, MAE: 60.01

[ ] feature_list = df.drop(columns=['target']).columns.tolist()
    subsets = get_all_subsets(feature_list)

    results = []
    n_train = X_train.shape[0]

[ ] for subset in subsets:
        X_train_sub = X_train[list(subset)]
        X_train_sub = sm.add_constant(X_train_sub)
        model = sm.OLS(y_train, X_train_sub).fit()
        aic = model.aic
        results.append({
            "features": subset,
            "aic": aic
        })

[ ] best = min(results, key=lambda x: x["aic"])
    best_features = list(best["features"])
    print("Best AIC subset:", best_features, "AIC =", best["aic"])

→ Best AIC subset: ['sex', 'bmi', 'bp', 's1', 's2', 's5'] AIC = 3829.5239090754194

[ ] X_test_best = X_test[best_features]
    X_test_best = sm.add_constant(X_test_best)
    y_test_pred = sm.OLS(y_train, sm.add_constant(X_train[best_features])).fit().predict(X_test_best)

[ ] mse_best = mean_squared_error(y_test, y_test_pred)
    mae_best = mean_absolute_error(y_test, y_test_pred)
    print(f"Best-subset Model → MSE: {mse_best:.2f}, MAE: {mae_best:.2f}")

→ Best-subset Model → MSE: 2846.29, MAE: 42.61
```

Figure 14: Enter Caption

8. (3 points) **1.5 Problem 6**

Computer Experiment

Use the following code to load the data:

```
from sklearn.datasets import load_wine
data = load_wine(as_frame=True)
df = data["frame"]
X = df["color_intensity"]
Y = df["hue"]
```

We can test for independence between X and Y using Pearson's correlation coefficient:

$$\rho = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{V(X)}\sqrt{V(Y)}}$$

We define the hypotheses:

$$H_0 : \text{X and Y are independent}$$

If X and Y are normally distributed, then under the null hypothesis, the t-statistic

$$T = \frac{\rho\sqrt{n-2}}{\sqrt{1-\rho^2}} \sim t_{n-2}$$

follows Student's t-distribution with $n-2$ degrees of freedom. So you can use the CDF of this distribution to find the p-value.

Find $\rho(X, Y)$ using the formula and compute the p-value from the CDF of the t-distribution. Do we reject the null hypothesis at $\alpha = 0.05$?

Solution:

```
[53] from sklearn.datasets import load_wine
     from scipy.stats import t, pearsonr

[54] data = load_wine(as_frame=True)
     df = data["frame"]
     X = df["color_intensity"]
     y = df["hue"]

[55] X.head()
```

	color_intensity
0	5.64
1	4.38
2	5.68
3	7.80
4	4.32

dtype: float64

```
[56] y.head()
```

	hue
0	1.04
1	1.05
2	1.03
3	0.86
4	1.04

dtype: float64

```
[57] n = len(X)

[58] x_bar = np.mean(X)
     y_bar = np.mean(y)

[59] numerator = np.sum((X - x_bar) * (y - y_bar))
     denominator = np.sqrt(np.sum((X - x_bar)**2) * np.sum((y - y_bar)**2))
```

Figure 15:

```
[60] rho = numerator / denominator

[61] T = rho * np.sqrt(n - 2) / np.sqrt(1 - rho**2)

[62] p_value = 2 * t.sf(np.abs(T), df=n - 2)

[63] print("Pearson correlation coefficient (ρ):", rho)
     print("t-statistic:", T)
     print("p-value:", p_value)

     Pearson correlation coefficient (ρ): -0.5218131932287576
     t-statistic: -8.115063490103632
     p-value: 8.075008429978501e-14

[64] alpha = 0.05
     if p_value < alpha:
         print("Reject H0 at α = 0.05")
     else:
         print("Do not reject H0 at α = 0.05")

     Reject H0 at α = 0.05
```

Figure 16:

You can also use the built-in function `scipy.stats.pearsonr` to find both the value of ρ and the p-value of this test:

```
from scipy.stats import pearsonr
corr, p_value = pearsonr(X, Y)
```

Do we reject the null hypothesis at $\alpha = 0.05$?

Solution:

```
Using Library:

[ ] rho_scipy, p_value_scipy = pearsonr(X, y)

[ ] print(f"Pearson correlation coefficient (ρ) = {rho_scipy}")
    print(f"p-value = {p_value_scipy}")

    Pearson correlation coefficient (ρ) = -0.5218131932287576
    p-value = 8.075008429978308e-14

[ ] if p_value_scipy < alpha:
        print("Reject H0 at α = 0.05")
    else:
        print("Do not reject H0 at α = 0.05")

    Reject H0 at α = 0.05
```

Figure 17:

9. (2 points) **1.6 Problem 7**

The Linnerud dataset consists of three exercise (data) and three physiological (target) variables collected from twenty middle-aged men in a fitness club.

- Generate a contingency table. For inspiration, refer to the Webinar 9 notebook.
- Test the hypothesis that X and Y are independent using `scipy.stats.chi2_contingency` at a confidence level of $\alpha = 0.05$.
- Do we reject the null hypothesis?

Solution:

```
[ ] from sklearn.datasets import load_linnerud
    import pandas as pd

[ ] data = load_linnerud(as_frame=True)
    df = data["target"]

[ ] data

    Show hidden output

[ ] data = load_linnerud(as_frame=True)
    df = data["target"]

[ ] X_cat = pd.qcut(df["Weight"], [0.0, 0.25, 0.5, 0.75, 1.0], labels=[0, 1, 2, 3])
    Y_cat = pd.qcut(df["Waist"], [0.0, 0.25, 0.5, 0.75, 1.0], labels=[0, 1, 2, 3])

[ ] obs = pd.concat([X_cat.rename("weight_bin"), Y_cat.rename("waist_bin")], axis=1)
    obs_table = obs.groupby(["weight_bin", "waist_bin"]).size().unstack(fill_value=0)

    <ipython-input-73-9edbf90a7c7b>:2: FutureWarning: The default of observed=False is
      obs_table = obs.groupby(["weight_bin", "waist_bin"]).size().unstack(fill_value=0

[ ] chi2_stat, pval, dof, expected = sts.chi2_contingency(obs_table)

[ ] print("Contingency Table: \n", obs_table)
    print("\nChi-Square Statistic:", chi2_stat)
    print("Degrees of Freedom:", dof)
    print("P-value:", pval)

    Contingency Table:
     waist_bin   0  1  2  3
    weight_bin
    0           4  1  0  0
    1           2  3  1  0
    2           0  1  3  0
    3           0  0  2  3

    Chi-Square Statistic: 23.41111111111111
    Degrees of Freedom: 9
    P-value: 0.005336151061769739

[ ] alpha = 0.05
    print("\n" + ("Rejected" if pval < alpha else "Not Rejected"))

    Rejected
```

Figure 18: